MeatScentinel Kiosk

MVP overview: context, kiosk function, ESP32 connection and Supabase connection

Repository: `meatscentinel`, folder `kiosk/` (branch `add-kiosk-app`) · October 2026

Summary

MeatScentinel checks whether raw chicken, pork or beef is still fresh. It combines two independent readings of the same sample: a **gas reading** (ammonia, NH_3, and hydrogen sulfide, H_2S, which rise as meat spoils) and an **image-based reading** from a partner mobile app. A shared Supabase database pairs the two by inspection ID and produces one final classification that inspectors review on a web dashboard.

The **kiosk** is the gas half. It is a touchscreen app that walks an operator through one inspection: choose the meat type, place the sample in the chamber, run a timed gas measurement, see a local Fresh / Spoiled verdict, upload the result, and ventilate the chamber for the next sample. The sensors and the fan live on an **ESP32** microcontroller that the kiosk controls over Wi-Fi; results go to **Supabase**, and are queued on the device whenever there is no internet.

Part	Technology	Role in the MVP
Kiosk app	Python 3.11, Kivy 2.3.1, SQLite	Operator UI, measurement logic, local verdict, upload with offline queue
Chamber hardware	ESP32 firmware SYS07_ESP32_Dashboard.ino	Weight sensor, two gas sensors, exhaust fan; served over HTTP
Cloud backend	Supabase (Postgres, RLS, triggers)	Stores submissions, pairs gas + image results, runs decision fusion
Web dashboard	React + Vite (web/)	Inspectors review records, add remarks, move case status
Mobile app	Partner team	Supplies the image-based result for the same inspection

> **MVP status (30 October 2026):** the kiosk ran three complete inspections against the real ESP32 (start, end, verdict, fan on, fan off). Supabase uploads have not yet been tested end to end, because the kiosk's `.env` credentials are not set; results were saved to the offline queue instead. See Section 6 for open items.

1. Context and system architecture

MeatScentinel is split across several pieces that each own one job. The kiosk never decides the final classification: it only contributes the gas half. Fusion, validation and access control live in the database so that no client can bypass them.

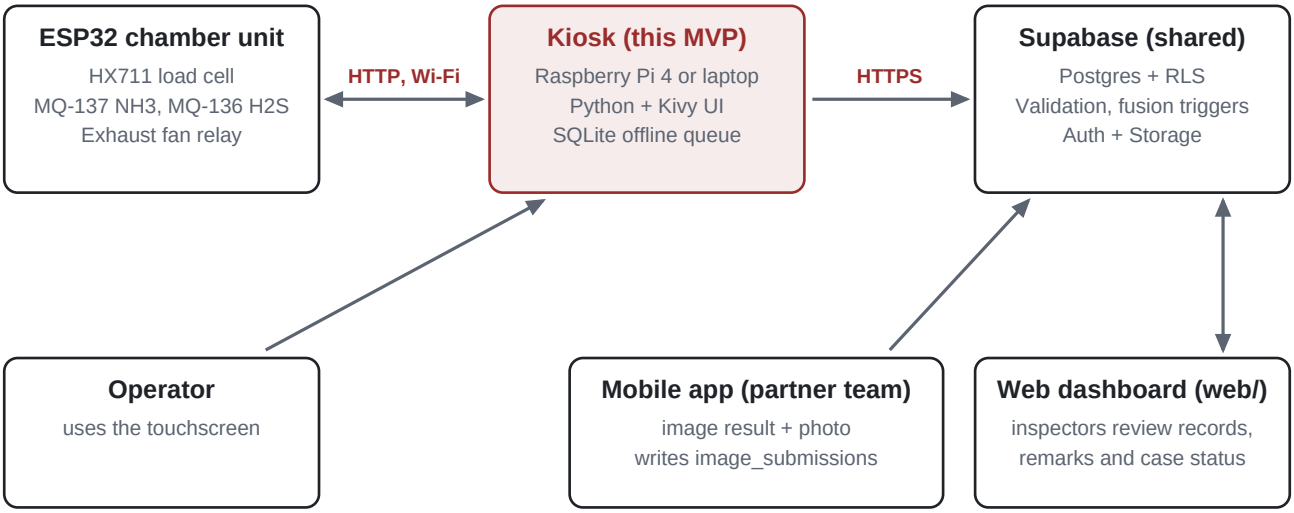
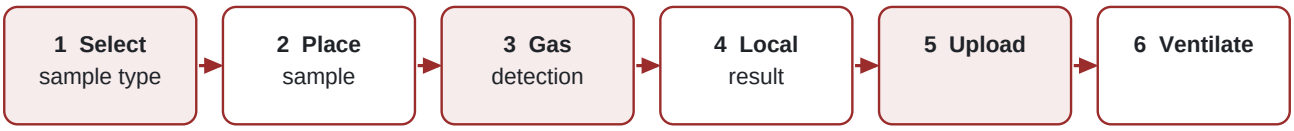


Figure 1. How the kiosk fits into MeatScentinel. The highlighted box is the subject of this document.

The repository also contains `device/`, an earlier device application that read the sensors directly from the Raspberry Pi (MCP3008 and HX711) and ships a simulator. The kiosk described here is the ESP32-based version and does not depend on it.

2. What the kiosk does

The kiosk is a single full-screen app with six screens, one per MVP module. Each inspection gets a new random inspection ID (a UUID) when the operator returns to the first screen.



#	Screen	What happens
1	Sample type	Operator picks Raw Chicken, Raw Pork or Raw Beef. The last choice is remembered across restarts. The choice selects that meat's thresholds.
2	Sample placement	Waits until the ESP32's weight sensor reports an object present. **Start Detection** stays disabled until then, and re-locks if the sample is removed. Shows "Scale unavailable" if the ESP32 cannot be reached.
3	Gas detection	Tells the ESP32 to start measuring, waits 5 s for sensor warm-up, then samples NH_3 and H_2S every 2 s for 30 s while a progress ring fills. Ends the measurement on the ESP32 afterwards. If the sample is removed mid-run the ESP32 cancels and the kiosk returns to placement.
4	Result	Averages the readings and compares them with the meat's thresholds. Shows **Fresh**, **Spoiled** or **Invalid Reading** with the average ppm values.
5	Upload	Sends the result to Supabase in the background. Shows Upload Successful, or Upload Failed with "stored locally, will retry". The operator can move on either way.
6	Ventilation	Turns on the exhaust fan for 30 s (the ESP32 also stops it on its own after 30 s), with a Stop button. Then **New Inspection** returns to screen 1.

How the verdict is decided

A reading is **Invalid** if fewer than 8 readings were collected, or if either gas's readings were too unstable (standard deviation above 15 ppm). Otherwise the sample is **Spoiled** if either average exceeds its limit, and **Fresh** if both are within limits. Invalid readings are still uploaded, marked invalid, so the database routes the inspection to For Review.

Meat type	NH_3 limit	H_2S limit
Raw Chicken	25 ppm	5 ppm
Raw Pork	30 ppm	6 ppm
Raw Beef	35 ppm	7 ppm

Thresholds are configured in `kiosk/thresholds.json`; timings and validity rules in `kiosk/config.json`.

Reliability features

- **Offline queue:** failed uploads are stored in a local SQLite database and retried every 30 s once the internet is reachable.
- **Global error screen:** unexpected errors show an error screen instead of crashing to the desktop.
- **Boot automation:** on the Raspberry Pi a systemd service starts the kiosk full-screen at boot with no desktop shown.
- **Logging:** every ESP32 command and upload is written to a rotating log, `inspection.log` (5 MB, 3 backups).

3. ESP32 connection

All chamber hardware is wired to an ESP32, not to the Raspberry Pi. The ESP32 firmware (`SYS07_ESP32_Dashboard.ino`) runs a small web server; the kiosk talks to it with plain HTTP requests over Wi-Fi. The address is set in `config.json` as `esp32.base_url` and is currently `http://192.168.4.1`, the ESP32's own Wi-Fi hotspot, so the kiosk machine must be joined to that network.

Request	Purpose	ESP32 rule
GET /api	Status snapshot as JSON (polled every 0.5 s)	Always allowed
POST /start	Begin a measurement	Refused (409) unless the fan is off, the scale is ready and an object is present
POST /end	Finish the measurement	Refused (409) unless a measurement is active
POST /fan/on	Start ventilation	Refused (409) unless a measurement was started and ended; 403 if relay control is disabled
POST /fan/off	Stop ventilation	Always allowed (403 only if relay control is disabled)

Example status response, as received from the ESP32 during testing:

```
{"mq136":640, "mq137":473, "hxRaw":9, "hxReady":true, "present":false, "measuring":false,
"completed":false, "fanOn":false, "relayEnabled":true}
```

Field	Meaning
mq137 / mq136	Raw 12-bit readings (0–4095) from the NH_3 and H_2S sensors
hxRaw, hxReady, present	Load-cell value, whether the scale is ready, and whether an object is on it

Field	Meaning
measuring, completed	Whether a measurement is running, and whether the last one was ended
fanOn, relayEnabled	Exhaust fan state, and whether the firmware allows fan control

How the kiosk uses it

- A background thread polls `GET /api` and caches the latest snapshot, so screens never wait on the network. A snapshot older than 3 s counts as "unavailable".
- Commands are sent from worker threads in the order the firmware requires: **start, end, fan on, fan off**. Each request times out after 2 s.
- The ESP32 reports raw readings, not ppm. The kiosk converts them with a linear formula: ppm = raw / 4095 × scale + offset, with scale 60 for NH_3 and 15 for H_2S. For example, the readings above give about 6.9 ppm NH_3 and 2.3 ppm H_2S.
- There is no simulated hardware. If the ESP32 is off or unreachable, the kiosk shows "unavailable" and will not start a measurement, so it can never record made-up readings.
- If the "end measurement" request is lost to a network drop, the ventilation screen ends the measurement again before turning on the fan, so ventilation is not blocked.

4. Supabase connection

The kiosk shares one Supabase project with the web dashboard. The database schema is owned and migrated from `supabase/migrations/`; the kiosk only writes to it. It connects with the Python `supabase` library, using two settings in `kiosk/.env`: `SUPABASE_URL` and `SUPABASE_SERVICE_KEY`. The service key is a server-side secret: the database grants the device write access through it, it must never be placed in the web app, and `.env` must never be committed.

What the kiosk writes, per inspection

Order	Table	Operation	Data
1	inspection_records	Upsert (insert, or no-op if it exists)	inspection_id only. A trigger creates the case's first status entry, "Open".
2	gas_submissions	Insert, only if no row exists yet	inspection_id, sample_type, nh3_ppm, h2s_ppm, gas_result (Fresh / Spoiled, empty if invalid), is_valid, detected_at

This order and these operations make every upload safe to retry. The parent record must exist first because `gas_submissions` references it, and gas submissions are immutable once received, so the kiosk checks for an existing row instead of overwriting it. A retry after a partial failure therefore never creates a duplicate or gets rejected.

What happens in the database

When both the gas submission (from the kiosk) and the image submission (from the mobile app) exist for the same inspection ID, a database trigger runs **decision fusion** once and writes a permanent final classification. Until both halves arrive, the record shows as Pending.

Gas result + image result	Final classification
Either side missing, invalid or incomplete	For Review
Either side Spoiled	Spoiled
Both Fresh	Fresh

- Row Level Security controls who can read what. Inspectors read gas submissions through the web dashboard; administrators manage accounts and cannot read inspection data.
- The kiosk never writes `status_entries`, `remarks`, `image_submissions` or `final_classification`; those belong to the web app, the mobile app and the fusion trigger.
- Realtime confirmation is intentionally inactive: the synchronous Supabase client the kiosk uses does not support it, and the upload's own success or failure already drives the Upload screen.

5. Running the kiosk

Step	Laptop (testing)	Raspberry Pi (deployment)
Install	`pip install -r kiosk/requirements.txt` in a Python 3.11 venv	Same, inside a venv on the Pi
Credentials	Copy `.env.example` to `.env` and fill in the Supabase URL and service key	Same
Network	Join the ESP32's Wi-Fi; check `http://192.168.4.1/api` in a browser	Pi must reach both the ESP32 and the internet
Start	`cd kiosk`, then `..\venv\Scripts\python main.py`; Esc quits	`deploy/install_service.sh` installs the boot service

6. MVP status and open items

Area	Status	Notes
Kiosk flow, all six screens	**Working**	Three full inspections completed against the real ESP32 on 30 Oct 2026.
ESP32 control	**Working**	Start, end, fan on and fan off all accepted by the firmware.
Supabase upload	**Not yet tested**	Needs `kiosk/.env`. Test results are waiting in the offline queue.
Wi-Fi link to ESP32	**Unstable**	About 48 dropped status requests in 10 minutes during testing; the laptop also kept switching back to another Wi-Fi network.
Internet while on ESP32 Wi-Fi	**Limitation**	The ESP32's hotspot has no internet, so uploads wait in the queue. A shared router or phone hotspot for both devices avoids this.
Gas ppm calibration	**Placeholder**	The raw-to-ppm formula is not calibrated. ppm values and verdicts are not accurate until bench-tested against reference gas.
Thresholds	**To validate**	Per-meat limits in thresholds.json need confirmation against the study's reference values.
Image submissions (V-17)	**Open**	How the mobile app writes to Supabase is still pending with the partner team; not a kiosk dependency.

Recommended next steps

- Add Supabase credentials to `kiosk/.env` and confirm one inspection appears on the web dashboard.
- Put the kiosk and ESP32 on one network that also has internet, and recheck connection stability.
- Calibrate the MQ-137 and MQ-136 sensors and replace the placeholder ppm formula.
- Confirm the per-meat thresholds, then run a test series with known fresh and spoiled samples.